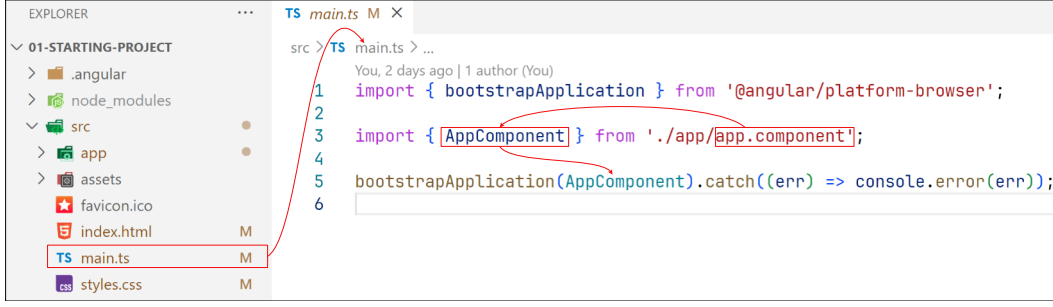


Maximilian Angular the Complete Guide 2025

1-Essentials

main.component.ts

1

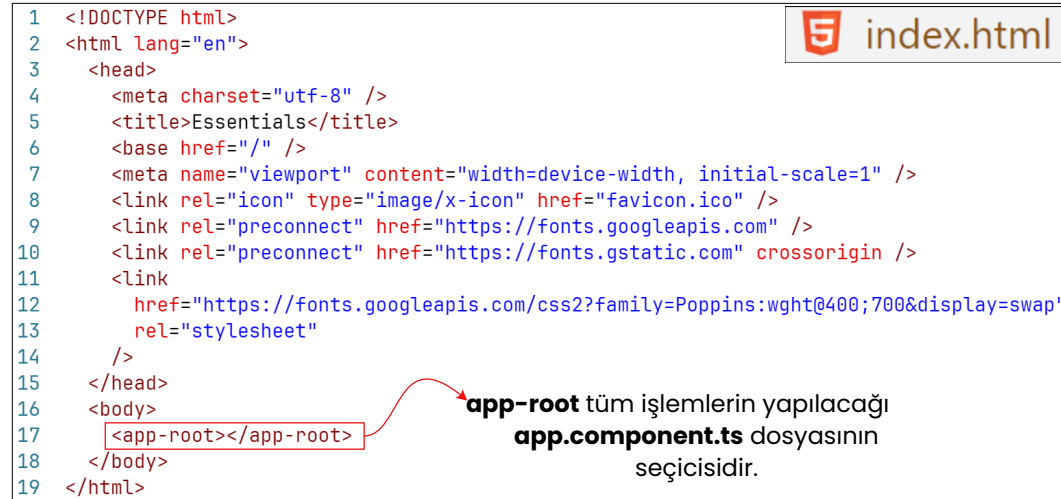


main.ts dosyası angularda tüm işlemlerin yapıldığı **app.ts** dosyasını çalıştıran başlangıç dosyasıdır. main.ts tek görevi dosyasının app.ts dosyasını çalıştırmaktır.



Angular temelde **TypeScript**, **html** ve **css** dosyalarından oluşan üçlü bir dosya yapısı kullanmaktadır. Angular, Ts(typescript) dosyasında bulunan talimatlar doğrultusunda html ve css dosyasını birleştirerek kullanıcı için işlevsel tek bir html dosyası üretir. Ts dosyasının kullandığı, html dosyası template denen görsel düzeni oluşturur. css dosyası html etiketlerinin görüntüsünü düzenler.

Src(root) klasörünün içinde bulunan **main.ts**, **index.html** ve **styles.css** global ayarları barındırır. main.ts dosyası tüm kodlamanın barındırdığı app.ts dosyasını işletir. index.html ana sayfayı oluşturur. styles.css en temel reset css gibi global css bilgilerini tutar.



Angular günümüzün popüler frontend çalışma düzeni olan single page application mantığına göre çalışır.

Angular 20 Öncesinde dosya isimleri altta ki gibi yapılmaktaydı;

app.component.css
app.component.html
app.component.ts

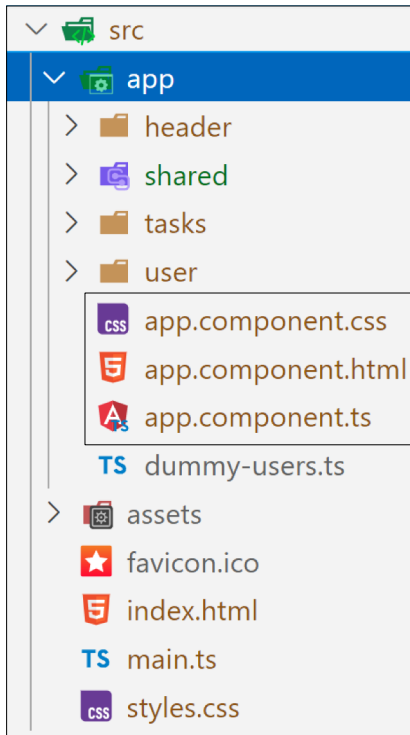
Angular 20 sonrasında bu zorunluluk kalktı. Dosya isimleri altta görüldüğü gibi daha sade şekilde yazılabilir.

app.css
app.html
app.ts

Ben notlarımda sürekli component yazmaktan kaçınacağım.

1-Essentials

app.component.ts



app.component tüm dosyaların çalıştığı **application**(uygulama) dosyasıdır.

@Component() fonksiyonu **Decorator** isimli konfigürasyon fonksiyonudur. @Component() dekoratörü belirtilen konfigürasyona göre AppComponent sınıfını ayarlar.

- **selector: "app-root"** : bu ayar app.component'inin bir üstte tanımlandığı html içinde nasıl seçileceğini ayarlar. Bir önceki sayfada belirtilen index.html dosyasında <app-root></app-root> elementi app.component'inin seçilerek yükleneceği elementi gösterir.
- **standalone:true** : bu ifade angular 19'dan sonra varsayılan olarak bulunduğundan kaldırılacaktır.
- **templateUrl: "/app.component.html"** : bu ayar app.component'inin tanımlandığı html template dosyasını belirtir.
- **styleUrl: "/app.component.css"** : bu ayar app.component'inin css dosyasını belirtir.
- **imports: [HeaderComponent, HeaderComponent, TasksComponent]** : app.component'inin içinde kullanılacak diğer komponentlerin import ayarıdır.

 app.component.ts

```
1 import { Component } from '@angular/core';
2
3 import { HeaderComponent } from '../header/header.component';
4 import { UserComponent } from '../user/user.component';
5 import { TasksComponent } from '../tasks/tasks.component';
6 import { DUMMY_USERS } from '../dummy-users';
7
8 @Component({
9   selector: 'app-root',
10  standalone: true,
11  templateUrl: './app.component.html',
12  styleUrl: './app.component.css',
13  imports: [HeaderComponent, UserComponent, TasksComponent],
14 })
15 export class AppComponent {
16   users = DUMMY_USERS;
17   selectedUserId?: string;
18
19   get selectedUser() {
20     return this.users.find((user) => user.id === this.selectedUserId);
21   }
22
23   onSelectUser(id: string) {
24     this.selectedUserId = id;
25   }
26 }
```

 app.component.css

```
1 main {
2   width: 90%;
3   max-width: 50rem;
4   margin: 2.5rem auto;
5   display: grid;
6   grid-auto-flow: row;
7   gap: 2rem;
8 }
9
10 #users {
11   list-style: none;
12   margin: 0;
13   padding: 0;
14   display: flex;
15   gap: 0.5rem;
16   overflow: auto;
17 }
18
```

 app.component.html

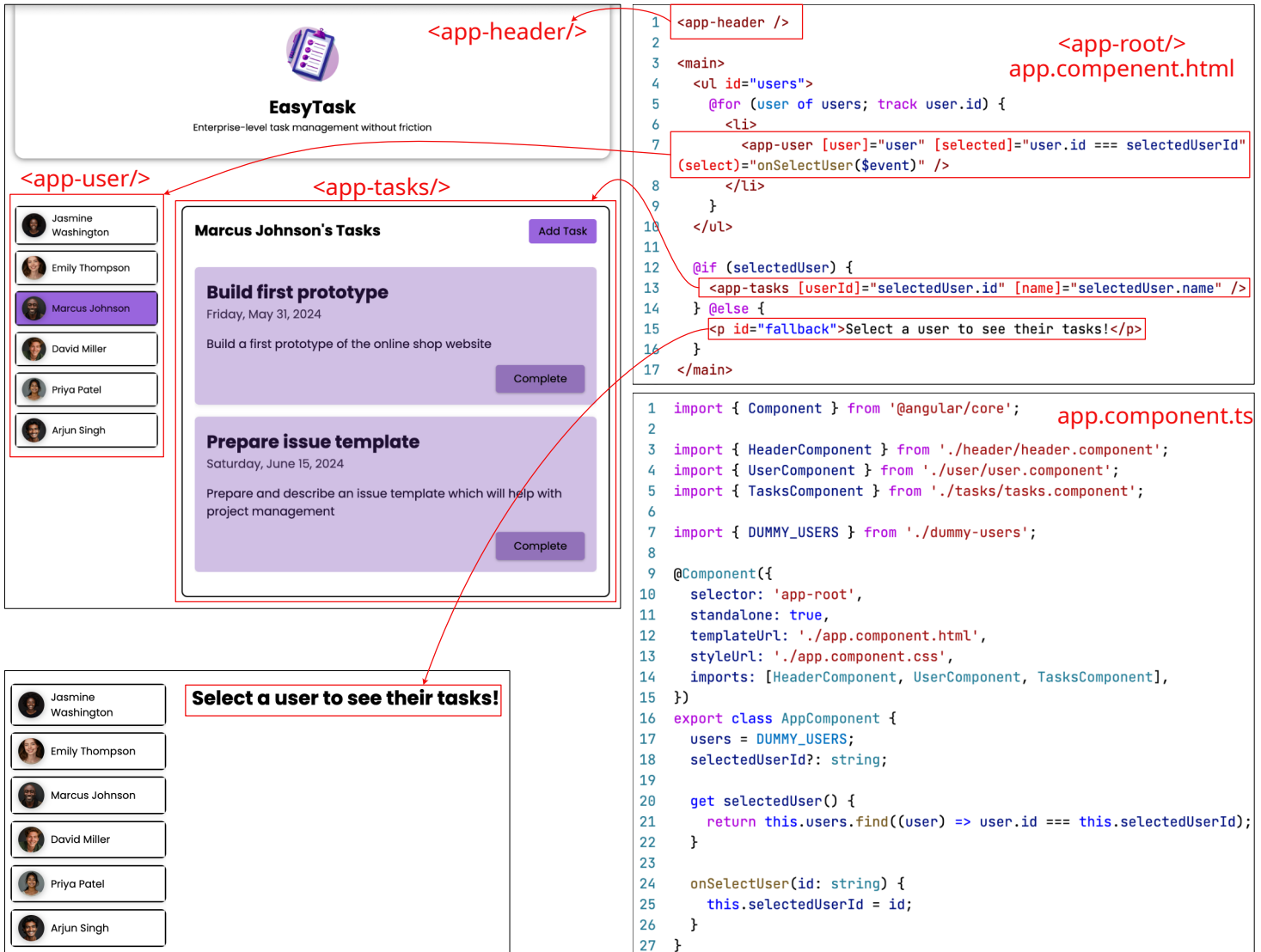
```
1 <app-header />
2
3 <main>
4   <ul id="users">
5     @for (user of users; track user.id) {
6       <li>
7         <app-user [user]="user" [selected]="user.id === selectedUserId" (select)="onSelectUser($event)" />
8       </li>
9     }
10  </ul>
11
12  @if (selectedUser) {
13    <app-tasks [userId]="selectedUser.id" [name]="selectedUser.name" />
14  } @else {
15    <p id="fallback">Select a user to see their tasks!</p>
16  }
17 </main>
```

Maximilian Angular the Complete Guide 2025

1-Essentials

app.component

3



The screenshot displays the EasyTask application interface and its corresponding code. The interface is divided into three main sections: a header, a user list, and a task list. The header section, labeled `<app-header />`, contains the 'EasyTask' logo and tagline. The user list section, labeled `<app-user />`, shows a list of users with 'Marcus Johnson' selected. The task list section, labeled `<app-tasks />`, displays tasks for the selected user, including 'Build first prototype' and 'Prepare issue template'. The code on the right shows the `app.component.html` and `app.component.ts` files. The `app.component.html` file uses Angular's `<app-header />`, `<app-user [user]='user' [selected]='user.id === selectedUserId' (select)='onSelectUser($event)' />`, and `<app-tasks [userId]='selectedUser.id' [name]='selectedUser.name' />` directives to render the application. The `app.component.ts` file imports the necessary components and defines the `AppComponent` class, which manages the `users` array and the `selectedUserId` property.

```
1 <app-header />
2
3 <main>
4   <ul id="users">
5     @for (user of users; track user.id) {
6       <li>
7         <app-user [user]="user" [selected]="user.id === selectedUserId"
8           (select)="onSelectUser($event)" />
9       </li>
10    }
11  </ul>
12
13  @if (selectedUser) {
14    <app-tasks [userId]="selectedUser.id" [name]="selectedUser.name" />
15  } @else {
16    <p id="fallback">Select a user to see their tasks!</p>
17  }
18 </main>
```

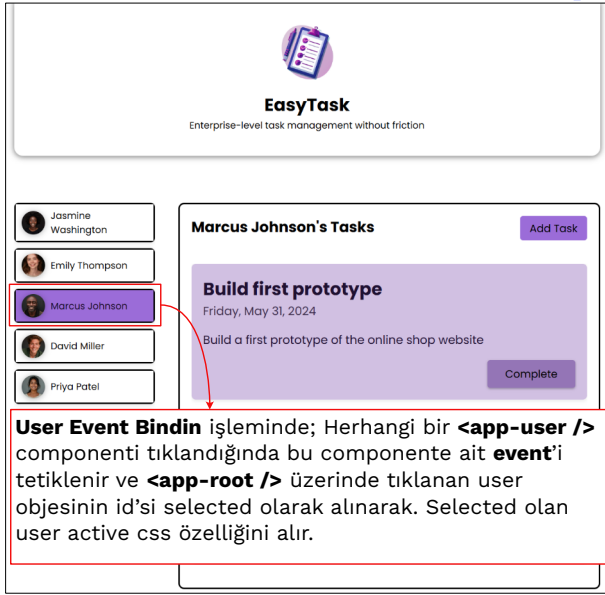
```
1 import { Component } from '@angular/core';
2
3 import { HeaderComponent } from './header/header.component';
4 import { UserComponent } from './user/user.component';
5 import { TasksComponent } from './tasks/tasks.component';
6
7 import { DUMMY_USERS } from './dummy-users';
8
9 @Component({
10   selector: 'app-root',
11   standalone: true,
12   templateUrl: './app.component.html',
13   styleUrls: ['./app.component.css'],
14   imports: [HeaderComponent, UserComponent, TasksComponent],
15 })
16 export class AppComponent {
17   users = DUMMY_USERS;
18   selectedUserId?: string;
19
20   get selectedUser() {
21     return this.users.find((user) => user.id === this.selectedUserId);
22   }
23
24   onSelectUser(id: string) {
25     this.selectedUserId = id;
26   }
27 }
```

app.component.html kendi içinde **Header**, **User** ve **Tasks** componentlerini barındırır.

- **<app-header />** sayfanın en başında bulunan başlık alanıdır. Sadece görüntüden oluşan fonksiyonsuz bir component olduğu için notu tutulmayacak.
- **<app-user />** bir kullanıcının buton olarak görüntülenmesini sağlayan bir componenttir. User component app component yüklendiğinde her kullanıcı için dinamik olarak üretilir. Kullanıcılardan birine tıklandığında bu kullanıcı aktif olarak işlenir ve bu kullanıcı ile ilgili **Tasks** listesi açılır.
- **<app-tasks />** aktif kullanıcı için açılan kullanıcının görevlerini içeren componenttir. Tasks Component seçili kullanıcının id ve kullanıcı adını parametre olarak alır.

Maximilian Angular the Complete Guide 2025 4

1-Essentials <app-user /> Event Binding



app.ts AppComponent sınıfını barındırır. Bu sınıf import ettiği **UserComponent** ve **TasksComponent** sınıfları üzerinde işlem yapar. AppComponent tüm kullanıcıların listesini **users** dizisinde tutar. Kullanıcının tıkladığı kullanıcı id'si **selectedUserId** değişkeninde tutulur. **selectedUser()** metodu seçilmiş kullanıcının user nesnesini döndürür. **onSelectedUser()** metodu tıklanan kullanıcının id'sini selectedUserId değişkenine atar.

user.ts UserComponent sınıfını üzerinden işlem yapar. UserComponent sınıfı dışarıdan **@Input()** tip decoratorü yardımıyla **user objesi** ve **selected boolean** değişkenini alır.

AppComponent sınıfı içinde **select** nesnesi **@Output()** decoratorü ile konfigüre edilerek **EventEmitter<>** jenerik sınıfından örneklenir. select nesnesi event mekanizmasının teteklenmesini sağlayan **.emit()** metodunu kullanır. Özet olarak select nesnesi @Output() ile ayarlanarak komponent dışına bilgi gönderen event binding objesidir.

```
1 import { Component } from '@angular/core';
2
3 import { HeaderComponent } from './header/header.component';
4 import { UserComponent } from './user/user.component';
5 import { TasksComponent } from './tasks/tasks.component';
6
7 import { DUMMY_USERS } from './dummy-users';
8
9 @Component({
10   selector: 'app-root',
11   standalone: true,
12   templateUrl: './app.component.html',
13   styleUrls: ['./app.component.css'],
14   imports: [HeaderComponent, UserComponent, TasksComponent],
15 })
16 export class AppComponent {
17   users = DUMMY_USERS;
18   selectedUserId?: string;
19
20   get selectedUser() {
21     return this.users.find((user) => user.id === this.selectedUserId);
22   }
23
24   onSelectUser(id: string) {
25     this.selectedUserId = id;
26   }
27 }
```

4-assagn selectedUserId

app.ts

```
1 <app-header />
2
3 <main>
4   <ul id="users">
5     @for (user of users; track user.id) {
6       <li>
7         <app-user
8           [user]="user"
9           [selected]="user.id === selectedUserId"
10          (select)="onSelectUser($event)"
11        />
12      </li>
13    }
14  </ul>
15
16  @if (selectedUser) {
17    <app-tasks [userId]="selectedUser.id" [name]="selectedUser.name" />
18  } @else {
19    <p id="fallback">Select a user to see their tasks!</p>
20  }
21 </main>
```

3-catch event

Fırlatılan user.id \$event ile yakalanır

app.html

```
1 <app-card>
2   <button [class.active]="selected" (click)="onSelectUser()"
3   <img [src]="imagePath" [alt]="user.name" />
4   <span>{{ user.name }}</span>
5   </button>
6 </app-card>
```

1-click event

user.html

```
1 button:hover,
2 button:active,
3 .active {
4   background-color: #9965dd;
5   color: #150722;
6 }
```

User.CSS

Adım adım Kullanıcının tıklanarak aktif olma durumunu inceleyelim;

- Her hangi bir kullanıcı kutucuğuna tıkladığında user.html üzerindeki **(click)="onSelectUser()** ifadesindeki **(click)** event'i user.ts üzerinde buluna **onSelectUser()** metodunu çalıştırır.
- user.ts onSelectUser() metodu **this.select.emit(this.user.id)** ile üzerinde barındırdığı **user.id** string değerini bir üstteki app.ts component'ine fırlatır.
- app.html üzerinde bulunan **(select)="onSelectUser(\$event)"** ifadesi üzerinde bulunan **\$event** değişkeni fırlatılan user.id değerini yakalar.
- yakalanan user.id değeri app.ts onSelectUser() metodu ile seçili user id değeri app.ts üzerindeki **this.selectedUserId** değişkenine atanır.

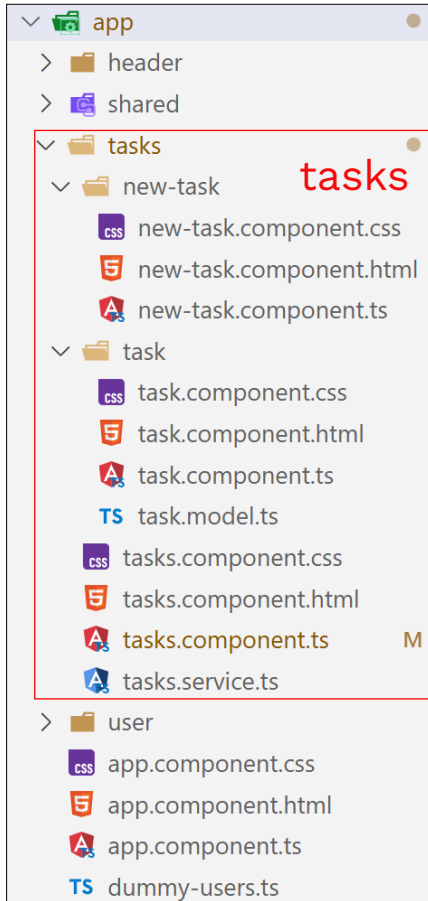
```
1 import { Component, Input, Output, EventEmitter } from '@angular/core';
2
3 import { type User } from './user.model';
4 import { CardComponent } from "../shared/card/card.component";
5
6 @Component({
7   selector: 'app-user',
8   standalone: true,
9   templateUrl: './user.component.html',
10   styleUrls: ['./user.component.css'],
11   imports: [CardComponent]
12 })
13 export class UserComponent {
14   @Input({ required: true }) user!: User;
15   @Input({ required: true }) selected!: boolean;
16   @Output() select = new EventEmitter<string>();
17
18   2-onSelectUser()
19   get imagePath() {
20     return 'assets/users/' + this.user.avatar;
21   }
22
23   onSelectUser() {
24     this.select.emit(this.user.id);
25   }
26 }
```

User nesnesinin user.id değeri fırlatılır.

user.ts

1-Essentials

Tasks Operations



```
1 import { Injectable } from '@angular/core';
2 import { type NewTaskData } from './task/task.model';
3
4 @Injectable({ providedIn: 'root' })
5 export class TasksService {
6   private tasks = [
7     {
8       id: 't1',
9       userId: 'u1',
10      title: 'Master Angular',
11      summary:
12        'Learn all the basic and advanced features of Angular &
13        how to apply them.',
14      dueDate: '2025-12-31',
15    },
16  ];
17
18  constructor() {
19    const tasks = localStorage.getItem('tasks');
20
21    if (tasks) {
22      this.tasks = JSON.parse(tasks);
23    }
24  }
25
26  getUserTasks(userId: string) {
27    return this.tasks.filter((task) => task.userId === userId);
28  }
29
30  addTask(taskData: NewTaskData, userId: string) {
31    this.tasks.unshift({
32      id: new Date().getTime().toString(),
33      userId: userId,
34      title: taskData.title,
35      summary: taskData.summary,
36      dueDate: taskData.date,
37    });
38    this.saveTasks();
39  }
40
41  removeTask(id: string) {
42    this.tasks = this.tasks.filter((task) => task.id !== id);
43    this.saveTasks();
44  }
45
46  private saveTasks() {
47    localStorage.setItem('tasks', JSON.stringify(this.tasks));
48  }
49 }
```

```
1 export interface Task {
2   id: string;
3   userId: string;
4   title: string;
5   summary: string;
6   dueDate: string;
7 }
8
9 export interface NewTaskData {
10  title: string;
11  summary: string;
12  date: string;
13 }
```

Tasks verilerini yönetmek için **tasks.service.ts** sınıfı kullanılır.

@Injectable({providedIn:"root"}) dekoratörü **TasksService** sınıfının **root** düzeninde tüm componentler tarafından erişilebileceği ayarını yapar.

TasksService sınıfı mini bir state manager gibi düşünülebilir. Bu service Tasks'lar ile ilgili işlemlerin gerekli componentler tarafından işlenmesine izin verir.

Injectable olması bu sınıfın root içinde bir defa oluşturulması ve erişilen componentler tarafından bir kaynak üzerinden düzenlenmesi anlamına gelmektedir. TasksService sınıfı her component için yeni ve farklı bir nesne olarak örneklenmez.

TasksComponent sınıfı;

1. app.html üzerinden **<app-tasks />** etiketinde bulunan **userId** ve **name** input değişkenleri aracılığıyla kullanıcı bilgilerini alır.
2. TasksComponent sınıfının constructor metodunda **TasksService** TasksComponent sınıfı ile ilişkilendirilir.
3. Yeni Task girişini için açılacak New Task modal ekranını yönetmek için TasksComponent sınıfının içinde **isAddingTask** değişkeni kullanılır.
4. task.component.html üzerinde tanımlanan **close** ve **click** event'ler TasksComponent üzerinde bulunan **onCloseAddTask** ve **onStartAddTask** metodları ile ilişkilendirilir.
5. tasks.html üzerinde user task listesi **selectedUserTasks** metodu yardımıyla tasksService kullanılarak alınır.

```
1 <app-header />
2
3 <main>
4   <ul id="users">
5     @for (user of users; track user.id) {
6       <li>
7         <app-user
8           [user]="user"
9           [selected]="user.id === selectedUserId"
10          (select)="onSelectUser($event)"
11        />
12      </li>
13    }
14  </ul>
15
16  @if (selectedUser) {
17    <app-tasks [userId]="selectedUser.id" [name]="selectedUser.name" />
18  } @else {
19    <p id="fallback">Select a user to see their tasks!</p>
20  }
21 </main>
```

```
1 @if (isAddingTask) {
2   <app-new-task [userId]="userId" (close)="onCloseAddTask()" />
3 }
4
5 <section id="tasks">
6   <header>
7     <h2>{{ name }}'s Tasks</h2>
8     <menu>
9       <button (click)="onStartAddTask()">Add Task</button>
10    </menu>
11  </header>
12
13  <ul>
14    @for (task of selectedUserTasks; track task.id) {
15      <li>
16        <app-task [task]="task" />
17      </li>
18    }
19  </ul>
20 </section>
```

```
1 import { Component, Input } from '@angular/core';
2
3 import { TaskComponent } from './task/task.component';
4 import { NewTaskComponent } from './new-task/new-task.component';
5 import { TasksService } from './tasks.service';
6
7 @Component({
8   selector: 'app-tasks',
9   standalone: true,
10  templateUrl: './tasks.component.html',
11  styleUrls: ['./tasks.component.css'],
12  imports: [TaskComponent, NewTaskComponent],
13 })
14 export class TasksComponent {
15   @Input({ required: true }) userId!: string;
16   @Input({ required: true }) name!: string;
17   isAddingTask = false;
18
19   private tasksService: TasksService;
20
21   constructor(tasksService: TasksService) {
22     this.tasksService = tasksService;
23   }
24
25   // Üstte yapılan constructor yapısının kısa hali
26   // constructor(private tasksService: TasksService) {}
27
28   // tasksService bu şekildeki gibi kullanılabilir
29   // private tasksService = inject(TasksService);
30
31   get selectedUserTasks() {
32     return this.tasksService.getUserTasks(this.userId);
33   }
34
35   onStartAddTask() {
36     this.isAddingTask = true;
37   }
38
39   onCloseAddTask() {
40     this.isAddingTask = false;
41   }
42 }
```

1-Essentials

Task Operations

tasks.html üzerinde

tasks.ts(**TasksComponent** sınıfı) ile gönderilen tasks listesi for döngüsü ile yazdırılır.

TaskComponent sınıfı üzerinde görüntülenecek date tipindeki veriyi düzenleyen

DatePipe pipe sınıfı vardır. **task.html** üzerinde **|** ile kullanılan DatePipe tarih verisinin çok rahat bir şekilde istenen formatta görüntülenmesini sağlar.

TaskComponent sınıfı da **TasksService** servis sınıfını kullanır. Burada TasksService constructor yönteminden farklı bir şekilde inject edilerek kullanılıyor(yani yeni bir sınıf olarak örneklenmeden, app root üzerinde bulunan sınıf üzerine enjekte edilerek kullanmak).

Bir task'ın tamamlandığı **onCompleteTask** metodu ile yapıyor. **task.html** üzerinde bulunan Complete butonuna tıklandığında **onCompleteTask** tetikleniyor ve **taskService** yardımıyla bu görev siliniyor.

```
1 @if (isAddingTask) {
2   <app-new-task [userId]="userId" (close)="onCloseAddTask()" />
3 }
4
5 <section id="tasks">
6   <header>
7     <h2>{{ name }}'s Tasks</h2>
8   </header>
9   <menu>
10    <button (click)="onStartAddTask()">Add Task</button>
11  </menu>
12 </header>
13
14 <ul>
15   @for (task of selectedUserTasks; track task.id) {
16     <li>
17       <app-task [task]="task" />
18     </li>
19   }
20 </ul>
21 </section>
```

tasks.componen.html

```
7 @Component({
8   selector: 'app-tasks',
9   standalone: true,
10  templateUrl: './tasks.component.html',
11  styleUrls: ['./tasks.component.css'],
12  imports: [TaskComponent, NewTaskComponent],
13 })
14 export class TasksComponent {
15   @Input({ required: true }) userId!: string;
16   @Input({ required: true }) name!: string;
17   isAddingTask = false;
18
19   onStartAddTask() {
20     this.isAddingTask = true;
21   }
22
23   onCloseAddTask() {
24     this.isAddingTask = false;
25   }
26 }
```

TasksComponent

```
1 <app-card>
2   <article>
3     <h2>{{ task.title }}</h2>
4     <time>{{ task.dueDate | date:'fullDate' }}</time>
5     <p>{{ task.summary }}</p>
6     <p class="actions">
7       <button (click)="onCompleteTask()">Complete</button>
8     </p>
9   </article>
10 </app-card>
```

task.component.html

```
1 import { Component, Input, inject } from '@angular/core';
2 import { DatePipe } from '@angular/common';
3
4 import { type Task } from '../task.model';
5 import { CardComponent } from '../../shared/card/card.component';
6 import { TasksService } from '../tasks.service';
7
8 @Component({
9   selector: 'app-task',
10  standalone: true,
11  templateUrl: './task.component.html',
12  styleUrls: ['./task.component.css'],
13  imports: [CardComponent, DatePipe]
14 })
15 export class TaskComponent {
16   @Input({required: true}) task!: Task;
17   private tasksService = inject(TasksService);
18
19   onCompleteTask() {
20     this.tasksService.removeTask(this.task.id);
21   }
22 }
```

task.component.ts

```
1 <div class="backdrop" (click)="onCancel()"></div>
2 <dialog open>
3   <h2>Add Task</h2>
4   <form (ngSubmit)="onSubmit()">
5     <p>
6       <label for="title">Title</label>
7       <input type="text" id="title" name="title" [(ngModel)]="enteredTitle" />
8     </p>
9
10    <p>
11      <label for="summary">Summary</label>
12      <textarea
13        id="summary"
14        rows="5"
15        name="summary"
16        [(ngModel)]="enteredSummary"
17      ></textarea>
18    </p>
19
20    <p>
21      <label for="due-date">Due Date</label>
22      <input
23        type="date"
24        id="due-date"
25        name="due-date"
26        [(ngModel)]="enteredDate"
27      />
28    </p>
29
30    <p class="actions">
31      <button type="button" (click)="onCancel()">Cancel</button>
32      <button type="submit">Create</button>
33    </p>
34  </form>
35 </dialog>
```

new-task.html

```
1 import { Component, EventEmitter, Input, Output, inject } from '@angular/core';
2 import { FormsModule } from '@angular/forms';
3
4 import { TasksService } from '../tasks.service';
5
6 @Component({
7   selector: 'app-new-task',
8   standalone: true,
9   imports: [FormsModule],
10  templateUrl: './new-task.component.html',
11  styleUrls: ['./new-task.component.css'],
12 })
13 export class NewTaskComponent {
14   @Input({ required: true }) userId!: string;
15   @Output() close = new EventEmitter<void>();
16   enteredTitle = '';
17   enteredSummary = '';
18   enteredDate = '';
19   private tasksService = inject(TasksService);
20
21   onCancel() {
22     this.close.emit();
23   }
24
25   onSubmit() {
26     this.tasksService.addTask(
27       {
28         title: this.enteredTitle,
29         summary: this.enteredSummary,
30         date: this.enteredDate,
31         userId: this.userId
32       }
33     );
34     this.close.emit();
35   }
36 }
```

new-task.ts

NewTaskComponent sınıfının **@Output() close = new EventEmitter<void>()**; ifadesindeki **close @Output()** dekoratörü ile ayarlanmış **EventEmmitter** nesnesidir. Bu ifade sınıf içinde **this.close.emit()**; komutuyla komponent dışına yayınlanır(fırlatılır, emit edilen).

NewTaskComponent'ı dışında close emmitterini yakalayan sınıf hangi işlemi yaptıracağını kendisi belirleyecektir. Bizim örneğimizde **tasks.componen.html** üzerinde bulunan **(close)="onCloseAddTask()"** (**close**) emmitterini yaklar ve **TasksComponent.onCloseAddTask()** metodunu işler. Output() işlemini kodlama adımlarını yazalım;

1. Dışarıda event'i yakalayacağımız component olan **tasks.component.html** üzerinde, **<app-new-task [userId]="userId" (close)="onCloseAddTask()" />** elementi ile yakamak istenen event emit değişkenini ve onun dışarıda kullanacağı metodu **(close)="onCloseAddTask()"** ifadesiyle belirt.
2. Dışarıda emmitter ile kullanılacak **TasksComponent** sınıfı içinde **onCloseAddTask(){this.isAddingTask = false;}** metodu tanımla.
3. Event'i fırlatacak(yayınlayacak, emit edecek) sınıfın olan **NewTaskComponent** sınıfı içinde **@Output() close = new EventEmitter<void>()**; ifadesiyle emmitteri tanımla.
4. **NewTasksComponent** içinde close emmitter'in nerede kullanacaksın sınıfın içinde gerekli metodlarda **this.close.emit()** komutunu kullan.

new-task.html içinde tanımlanan **(click)="onCancel()"** event'i click eventi ile tetiklenir ve **onCancel()** metodunu çalıştırır. **onCancel()** metodu **this.close.emit()** komutunu işleterek close emmitterini yayımlar. **onSubmit()** metodunun sonunda da **this.close.emit()** kullanılarak emmitter yayınlanabilmektedir.

Two-Way Binding

new-task.html üzerinde bulunan **[(ngModel)]="enteredTitle"** şekilde tanımlanan değişkenler iki yönlü binding mekanizmasıyla çalışır.

[(ngModel)] ile belirtilen **enteredTitle**, **enteredSummary**, **enteredDate** değişkenleri iki yönlü çalışır. Kullanıcı bu form input alanlarına giriş yaptığı her klavye girişi ngModel ile belirtilen değişken üzerinde tutulur. submit tipinde olan Create butonuna tıklandığında form submission işlemi form etiketindeki **(ngSubmit)="onSubmit()"** ifadesiyle onSubmit metodunu tetikler ve kullanıcı girişi yapılır. onSubmit() metodunda tasks girişi yapıldıktan sonra **this.close.emit()** komutu çalıştırılarak new-task giriş formunun kapatılması tetiklenir.